

Homework 2

Computational Bootcamp, Summer 2026

Due: Wednesday, August 12

Submission: you will be added as a collaborator on your own private repository `hw2-<your-github-username>` in the course organization. Clone it, write your answers in `homework2.jl`, and `git push` to submit. Your results will be posted back as a GitHub issue. See the repository README for setup and the required function names.

Question 1: Labor supply

A worker with non-labor income y chooses hours $h \in [0, 1]$ to maximize

$$u(h) = \log(c) + \alpha \log(1 - h), \quad c = wh + y,$$

with $\alpha = 1.5$ and $y = 0.5$.

- A. Write `optimal_hours(w;  $\alpha = 1.5$ ,  $y = 0.5$ )`, returning optimal hours using `optimize` with Brent's method (remember: `Optim` minimizes, and $\log(0) = -\infty$, so shrink the interval slightly). For a grid of wages $w \in [0.5, 5]$, plot the labor supply curve $h^*(w)$.
- B. The interior solution can be found analytically from the first-order condition. Derive it and implement it as `hours_closed(w;  $\alpha = 1.5$ ,  $y = 0.5$ )`, clamped to $[0, 1]$. Confirm it agrees with part A. At what wage does the worker stop working, and why does that number make sense?
- C. Repeat part A with $y = 2.0$ and plot both supply curves. Interpret the difference in terms of income effects.

Question 2: Maximum likelihood for a binary choice

OLS has a closed-form formula. Most estimators do not: you write down an objective and hand it to an optimizer.

Individuals choose college ($d_i = 1$) if the latent value $v_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ is positive, where x_i is a (scalar) test score and $\varepsilon_i \sim \text{logistic}$, so that

$$P(d_i = 1 \mid x_i) = \Lambda(\beta_0 + \beta_1 x_i) = \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)}.$$

- A. Simulate a dataset with $N = 5,000$, $x_i \sim N(0, 1)$, $\beta_0 = -0.4$, $\beta_1 = 1.2$. (Hint: $d_i = 1$ if $\Lambda(\beta_0 + \beta_1 x_i) > u_i$ with $u_i \sim U(0, 1)$.)

- B. Write the log likelihood $\ell(\beta) = \sum_i [d_i \log \Lambda(\cdot) + (1 - d_i) \log(1 - \Lambda(\cdot))]$ and estimate β by MLE with `estimate_logit(x, d; method = :lbfgs)`, which starts from $\beta = (0, 0)$, takes `method = :nm` for Nelder–Mead or `:lbfgs` for LBFGS, and returns $(\hat{\beta}, \text{iterations})$. Optim finite-differences the LBFGS gradient by default, which is fine; you could optionally use `autodiff = :forward` for exact derivatives but this needs generic code (no type declaration `::Vector{Float64}`). Check whether both `nm` and `lbfgs` recover the truth and compare iteration counts.

Question 3: A hard test function

The Himmelblau function,

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2,$$

has multiple minima and is a classic optimizer stress test. Implement it as `himmelblau(v)`, taking a length-2 vector.

- Produce surface and contour plots over $x, y \in [-4, 4]$. How many local minima do there appear to be?
- Solve for the gradient and Hessian analytically and optimize using Newton’s method, i.e. iterate $x_{n+1} = x_n - H(x_n)^{-1} \nabla f(x_n)$. Write this as `newton_min(grad, hess, x0; tol = 1e-10, maxiter = 100)`, returning the minimizer. This is univariate Newton in many dimensions: dividing by f' becomes a linear solve, which in Julia is `hess(x) \ grad(x)`, never `inv(hess(x)) * grad(x)`. Run it from several starting points. How does the answer depend on the initial guess, and is every point it converges to a minimum? (Look at what the method is actually solving.)
- Now ignore the derivatives and optimize using Nelder–Mead from the same starting points. Compare the number of iterations and the minima found. What would you have concluded from a single run?

Question 4: Market equilibrium

Demand and supply in a market are

$$Q_d(p) = 2p^{-0.5}, \quad Q_s(p) = p^{0.7}.$$

- Write the excess demand function $Z(p) = Q_d(p) - Q_s(p)$ as `excess_demand(p)` and find the equilibrium price with (i) Newton’s method (derive Z' by hand), (ii) `bisection(f, a, b; tol = 1e-10, maxiter = 1000)`, which you write yourself and which returns `(root, iterations)`, on $[0.1, 10]$, and (iii) `find_zero` from `Roots.jl`. Confirm all three agree with the closed-form p^* solved by hand.
- Count iterations for Newton vs. bisection at tolerance 10^{-10} . Which converges faster, and why might you still prefer bisection? Find a starting guess from which Newton fails, and explain what went wrong.
- Economists rarely have Z' , but they usually have a story about how prices adjust: raise the price when demand exceeds supply. That is *tâtonnement*, a fixed point of $T(p) = p + \lambda Z(p)$, where $\lambda > 0$ is a dampening parameter applied to the adjustment rule $p \mapsto p + Z(p)$. Write

`tatonnement(p0; λ = 0.5, tol = 1e-10, maxiter = 10_000)`, iterating $p_{n+1} = T(p_n)$ and returning `(p, iterations)` (keep p positive; Q_d is undefined at $p \leq 0$). Run it from $p_0 = 1$ for $\lambda \in \{0.05, 0.5, 1.0, 2.5\}$, the last of which is outside the $(0, 1]$ range. Predict your answer: a differentiable T converges to p^* from nearby guesses when $|T'(p^*)| < 1$ and diverges for $|T'(p^*)| > 1$. Here $T'(p^*) = 1 + \lambda Z'(p^*)$.

Question 5: Interpolation and grid placement

We consider how to optimally approximate a function using linear interpolation. Let $f(x) = \ln(x + 1)$ on $x \in [0, 100]$.

- A. Write `interp_error(interior)`, which takes a vector of 9 interior grid points, appends the endpoints 0 and 100, evaluates f on the resulting 11-point grid, builds a linear interpolant $\hat{f}$, and returns the three vectors $(f(x), \hat{f}(x), |\hat{f}(x) - f(x)|)$ at $x = 0, 0.1, \dots, 100$. Also write `total_error(interior)`, the sum of those absolute errors. Start with an evenly spaced grid $(0, 10, 20, \dots)$, plot f , $\hat{f}$, and the error. Where is the error highest? Interpret.
- B. Write `optimal_grid()`, which uses Nelder–Mead started from the evenly spaced interior points to find the 9 points minimizing `total_error`, and returns them. (You will need to rule out negative or non-monotone grid points to keep Nelder–Mead from behaving stupidly — a large penalty value is the usual trick.) What are the selected points? By how much does the error fall? Plot the new f , $\hat{f}$, and errors, and relate your answer to the common practice of log-spacing asset grids.
- C. Now build a cubic spline on the same evenly spaced grid and report its total error. Comparing your three numbers (even + linear, even + cubic, optimized + linear): given a fixed budget of 11 points, would you rather spend your effort on a better interpolant or on better-placed points, and what is it about this f that makes that so?

Question 6: Expected utility via quadrature

A Monte Carlo estimate of an integral converges like $1/\sqrt{n}$. That rate is a fact about simulation, not about integration.

A household has CRRA utility $u(c) = c^{1-\gamma}/(1-\gamma)$ with $\gamma = 2$, and consumption is derived from a lognormal shock: $\log c \sim N(\mu, \sigma^2)$ with $\mu = 0$, $\sigma = 0.5$.

- A. Compute $E[u(c)]$ by Gauss–Hermite quadrature (`FastGaussQuadrature.jl`) with $n = 5, 9, 15$ nodes, as `eu_quadrature(n)`. Recall the change of variables: for $X \sim N(\mu, \sigma^2)$, $E[g(X)] \approx \frac{1}{\sqrt{\pi}} \sum_i w_i g(\mu + \sqrt{2}\sigma x_i)$.
- B. Compute the same expectation by Monte Carlo, as `eu_montecarlo(M; seed = 1)`, with $M = 100, 10,000$, and $1,000,000$ draws.
- C. The exact answer is a moment of a lognormal distribution: $E[c^k] = \exp(k\mu + k^2\sigma^2/2)$, so at $k = 1 - \gamma$, $E[u(c)] = \exp((1 - \gamma)\mu + (1 - \gamma)^2\sigma^2/2)/(1 - \gamma)$. Use it as the benchmark and report each method’s absolute error against its number of function evaluations. What do you conclude about quadrature vs. simulation for smooth, low-dimensional integrals?